

Chapter 1

Wildfire Costs Calculation Documentation

1.1 Introduction

The `wildfire_costs.py` script calculates expected wildfire costs using a probabilistic risk approach. It models ignition rates by terrain and construction type, event severity, and applies a growing annuity formula to account for increasing risk over the project lifetime.

1.2 Method Overview

1.2.1 Purpose

This script calculates wildfire costs by:

1. Calculating segment-specific ignition rates by terrain and construction type
2. Aggregating to total annual event rate (λ)
3. Computing Expected Annual Loss (EAL) = $\lambda \times \text{severity}$
4. Calculating nominal total cost using growing annuity (accounting for risk growth)
5. Computing present value using growing annuity formula

1.2.2 Calculation Approach

The wildfire cost calculation follows this structure:

$$f_{i,t} = \text{Base Rate}_{\text{terrain}} \times \text{Construction Multiplier} \quad (1.1)$$

$$\lambda_{\text{segment}} = \text{Miles}_{\text{terrain}} \times f_{i,t} \quad (1.2)$$

$$\lambda_{\text{total}} = \sum_{\text{terrains}} \lambda_{\text{segment}} \quad (1.3)$$

$$\text{EAL} = \lambda_{\text{total}} \times \text{Severity} \quad (1.4)$$

$$\text{Nominal Total} = \text{EAL} \times \frac{(1+g)^N - 1}{g} \quad (\text{if } g \neq 0) \quad (1.5)$$

$$\text{PV} = \text{EAL} \times \frac{1 - \left(\frac{1+g}{1+d}\right)^N}{d - g} \quad (\text{if } d \neq g) \quad (1.6)$$

The PV is discounted for the delay and construction period so the cost stream begins at Commercial Operation Date (COD), not year 1; see “Payment timing (COD)” below.

where:

- $f_{i,t}$ = effective ignition rate per mile for terrain i and construction type t
- λ = annual event rate (events/year)
- g = risk growth rate (annual increase in expected losses)
- d = discount rate
- N = project lifetime

1.2.3 Inputs and Outputs

Inputs:

- Project technical details (construction type)
- Physical details (terrain distribution)
- Wildfire parameters (ignition rates by terrain, severity, growth rate, construction multipliers)
- Discount rate source (social or wacc_real; societal perspective by default)
- Project lifetime

Outputs:

- Total annual event rate (lambda)
- Expected Annual Loss (EAL)
- Event rates by terrain segment
- Nominal total cost (growing annuity)
- Present value cost (growing annuity with discounting)

1.2.4 Integration with Other Scripts

This script integrates with:

- `smart_loaders.py` — `load_project_technical_details`, `load_physical_details`, `load_financing_data`, `load_wildfire_costs`, `get_physical_data_raw`, `get_financing_data_raw`, `load_terrain_miles`
- `constants.py` — `MIN_DISCOUNT_RATE`, `GROWTH_RATE_TOLERANCE`
- `financial_utils.py` — `get_discount_rate_from_config`, `calculate_growing_annuity_pv`, `validate_discount_rate`, `calculate_nominal_growing_series`
- `calculation_utils.py` — `build_category_string`, `normalize_construction_type_for_yaml`
- `smart_output.py` — `CTCCOutputManager` (`add_wildfire_costs`, `write_batch_summary`)

1.3 Key Functions

1.3.1 `get_discount_rate_from_config` (from `financial_utils`)

The script uses `financial_utils.get_discount_rate_from_config(config_yaml, financing_yaml, rate_key="discount_rate_source")` to get the discount rate. In `main()`, it is called with `wildfire_yaml`, `get_financing_data_raw()`, and `rate_key="discount_rate_source"`. Returns (`discount_rate`, `source_description`). Supported sources are `social` (societal perspective, default) and `wacc_real` (utility perspective).

1.3.2 `calculate_wildfire_costs(wildfire_yaml, construction_type, terrain_miles, project_lifetime, discount_rate, delay_years=0, construction_years=0)`

Calculates expected wildfire costs using segment-based ignition rates.

Parameters:

- `wildfire_yaml` (dict): Loaded wildfire YAML data
- `construction_type` (str): Construction type (overhead, underground, etc.)
- `terrain_miles` (dict): Dictionary of terrain type to miles
- `project_lifetime` (int): Project lifetime in years
- `discount_rate` (float): Discount rate for PV calculation
- `delay_years` (float): Delay years before construction (default 0)
- `construction_years` (float): Construction period years (default 0)

Returns:

- Dictionary containing:

- `lambda_total` (float): Total annual event rate
- `EAL` (float): Expected Annual Loss
- `lambda_by_terrain` (dict): Event rates by terrain segment
- `severity` (float): Mean uninsured loss per event (see “Severity definition” below)
- `growth_rate` (float): Risk growth rate
- `nominal_total` (float): Total nominal cost
- `pv_cost` (float): Present value cost

Key Logic:

```

1 # Calculate segment-specific ignition rates
2 for terrain, miles in terrain_miles.items():
3     if miles > 0:
4         base_rate = ignition_rates_by_terrain.get(terrain, 0.0)
5         f_i_t = base_rate * construction_multiplier
6         lambda_segment = miles * f_i_t
7         lambda_total += lambda_segment
8
9 # Calculate EAL
10 EAL = lambda_total * severity
11
12 # Calculate nominal total (growing annuity)
13 if abs(growth_rate) < 1e-9:
14     nominal_total = EAL * project_lifetime
15 else:
16     nominal_total = EAL * ((1 + growth_rate) ** project_lifetime -
17                             1) / growth_rate
18
19 # Calculate PV (growing annuity with discounting)
20 if abs(discount_rate - growth_rate) < 1e-9:
21     pv_cost = EAL * project_lifetime
22 else:
23     pv_cost = EAL * ((1 - ((1 + growth_rate) / (1 + discount_rate))
24                     ** project_lifetime)
25                     / (discount_rate - growth_rate))

```

1.3.3 main()

Main execution function that orchestrates the wildfire cost calculation and output.

Workflow:

1. Loads project technical details; constructs category via `build_category_string`; loads terrain via `load_terrain_miles()`; loads wildfire_yaml via `load_wildfire_costs()`; loads financing_yaml via `get_financing_data_raw()`

2. `discount_rate, discount_source = get_discount_rate_from_config(wildfire_yaml, financing_yaml, rate_key="discount_rate_source")`
3. Calls `calculate_wildfire_costs(wildfire_yaml, construction_type, terrain_miles, project_lifetime, discount_rate, delay_years=..., construction_years=...)`
4. Displays results; `add_wildfire_costs(results); write_batch_summary()`

1.4 Example

The following example demonstrates a typical usage of the wildfire cost calculation:

```

1 # Example: 500 MW Overhead AC line
2 # Category: "Overhead/AC/500MW/Advanced Aluminum/NA"
3 #
4 # Inputs (from YAML):
5 #   - Terrain distribution:
6 #     * Forested: 40 miles, base rate: 0.0001 events/mi/yr
7 #     * Farmland: 60 miles, base rate: 0.00005 events/mi/yr
8 #   - Construction multiplier: 1.0 (overhead baseline)
9 #   - Severity: $50M per event
10 #   - Risk growth rate: 2% per year
11 #   - Project lifetime: 50 years
12 #   - Discount rate: 3% (social)
13 #
14 # Calculation:
15 #   Forested segment: 40 * 0.0001 * 1.0 = 0.004 events/year
16 #   Farmland segment: 60 * 0.00005 * 1.0 = 0.003 events/year
17 #   Total lambda = 0.007 events/year
18 #   EAL = 0.007 * $50M = $350,000/year
19 #   Nominal total = $350k * ((1.02^50 - 1) / 0.02) = $28.5M
20 #   PV = $350k * ((1 - (1.02/1.03)^50) / (0.03 - 0.02)) = $12.3M
21 #
22 # Output:
23 #   - Total annual events: 0.007 events/year
24 #   - EAL: $350,000/year
25 #   - Nominal total: $28.5M
26 #   - Present value: $12.3M

```

Integration Example:

```

1 # When called from ctcc.py:
2 csv_manager = CTCCOutputManager()
3 # Results dict already has all needed values from
   calculate_wildfire_costs()
4 csv_manager.add_wildfire_costs(results)
5 csv_manager.write_batch_summary()

```

1.5 Key Implementation Details

1.5.1 Payment timing (COD)

The cost stream starts at Commercial Operation Date (COD), i.e. when the line is in operation (after delay and construction). Wildfire risk is treated as operating-period risk: the PV is discounted for the delay and construction period so the first year of expected loss is at COD, not at year 1. This matches the timing used for wildfire liability insurance (file 04).

1.5.2 Severity definition (uninsured residual)

Severity (`severity_per_event`) is the expected **uninsured** loss per wildfire event: deductibles, amounts above the liability limit, exclusions, or societal/external damages. The insured portion of wildfire risk is captured by the optional wildfire liability insurance module (file 04). Defining severity as uninsured ensures that including both this module and wildfire liability insurance does not double-count; the former is uninsured expected loss (societal or residual), the latter is the utility's cost to transfer insured risk.

1.5.3 Discount rate and perspective

The wildfire module uses the social discount rate (societal perspective) when `discount_rate_source` is `social` (the default). The resulting PV is a **societal** cost. Use `wacc_real` for a utility perspective if desired.

1.5.4 Probabilistic Risk Approach

The script uses a probabilistic approach where wildfire events are modeled as Poisson processes. The annual event rate (`lambda`) represents the expected number of events per year, and EAL represents the expected annual loss.

1.5.5 Construction Type Multipliers

Different construction types have different ignition rates. Overhead lines have the highest risk (multiplier = 1.0), while underground and subsea have much lower multipliers (e.g., 0.01 for underground).

1.5.6 Growing Annuity Formula

The script accounts for increasing wildfire risk over time using a growing annuity formula. This reflects the reality that climate change and other factors may increase wildfire frequency and severity.

1.5.7 Edge Cases

The script handles edge cases:

- If growth rate = 0: Uses simple annuity formula
- If discount rate = growth rate: Uses simplified formula ($PV = EAL \times N$)

1.5.8 AFUDC Treatment

Wildfire costs are probabilistic future losses, not capital costs. Therefore, they are not AFUDC-eligible. AFUDC applies only to construction work in progress.

1.5.9 Warning for High Growth Rates

If the risk growth rate exceeds the discount rate, the script issues a warning because the present value grows over time, indicating that risk is escalating faster than discounting.